

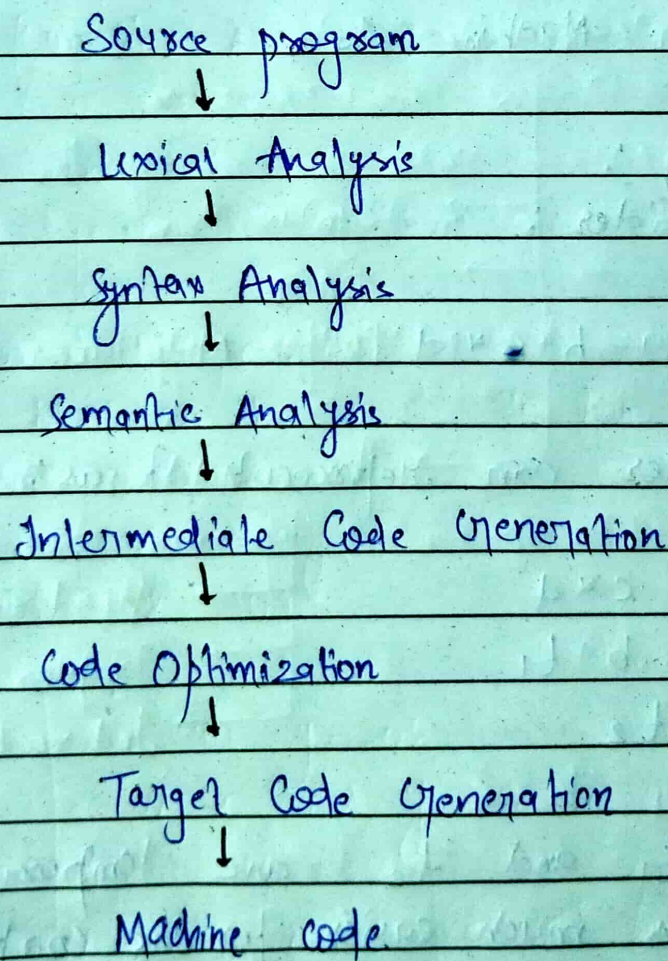
Phase 4:

→ Intermediate Code Generation :-

Intermediate Code Generation (ICG) is the compiler phase that converts the source program/syntax tree into an intermediate representation (IR) that is easier to translate into machine code.

It comes after semantic analysis and before code optimization and target code generation.

Compiler flow



→ What is intermediate code?

Intermediate code is a machine-independent representation of a program.

It is not the original source code and is not directly machine code.

High level language



Intermediate code



Machine code

for eg :

Consider :

$$a = b + c * d$$

A compiler can represent it as:

$$t_1 = c * d$$

$$t_2 = b + t_1$$

$$a = t_2$$

here, t_1 and t_2 are temporary variables. This form is much easier for the compiler to optimize and convert it into machine instⁿ.

→ Why is intermediate code needed?

Intermediate code provide a middle level b/w the source language and target machine.

Main Advantages:

1. Machine independence:

The intermediate code does not depend strongly on a particular processor.

The same intermediate representation can be translated into different target machines.

2. Easy optimization:

The compiler can optimize intermediate code before generating machine code.

Eg:

$$x = 10 \times 2$$

can be optimized to

$$x = 20$$

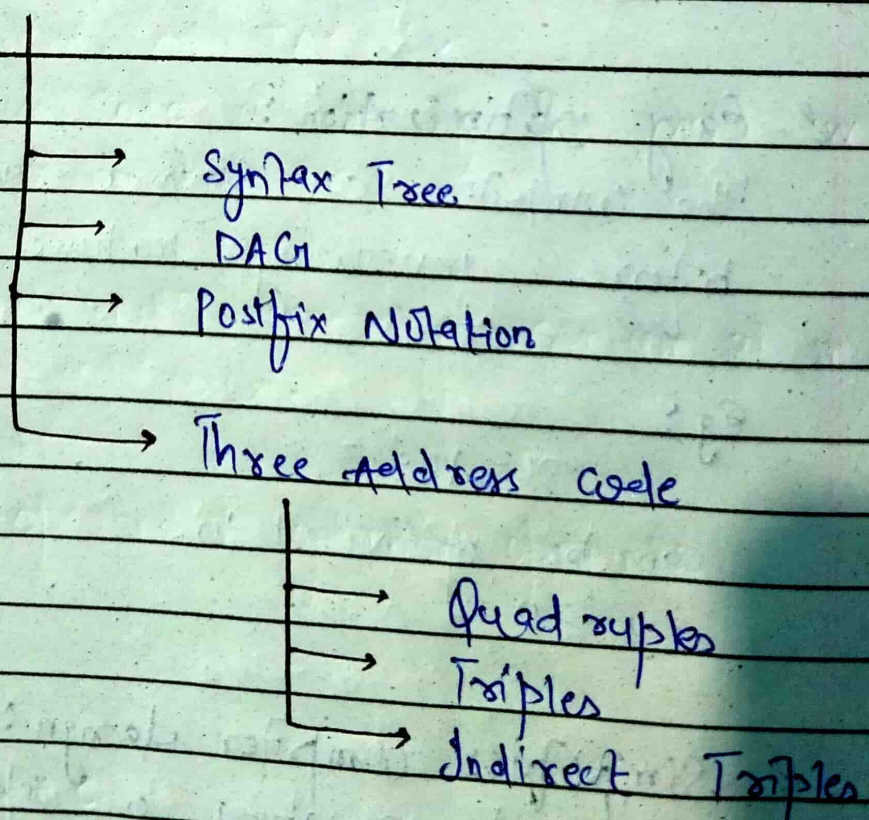
3. Simplifies compiler design:

Instead of creating a separate complete compiler for every source-target combination, the compiler can use an intermediate representation

- Properties of Good intermediate Code:
- A good intermediate representation should be:
- Easy to generate
 - Easy to understand
 - Easy to optimize
 - Machine independent
 - Easy to translate into target code
 - Compact and efficient

→ Common forms of intermediate Code:

Intermediate Code

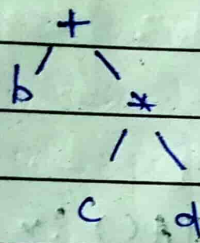


→ Syntax Tree as intermediate Representation:
A syntax tree represents the hierarchical structure of an expression or program.

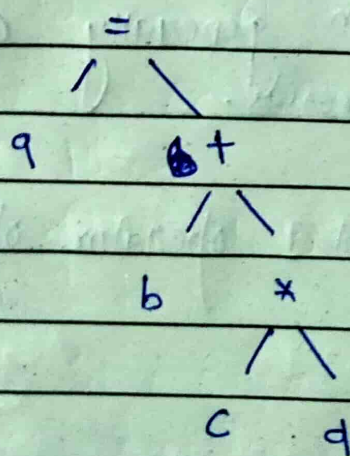
for:

$$a = b + c * d$$

The expression part can be represented as:



The assignment can be represented as:



→ Postfix Notation:

In postfix notation, the operator is written after its operands.

for:

$$a + b * c$$

$$a + bc *$$

$$abc * +$$

→ Three-Address Code (TAC):

Three-Address Code is one of the most important forms of intermediate code.

Each ~~operand~~ instrⁿ generally contain at most three addresses.

$$\text{result} = \text{operand 1} \text{ operator operand 2}$$

Eg:

$$t_1 = b * c$$

here,

$t_1 \rightarrow \text{result}$
 $b \rightarrow \text{operand 1}$
 $c \rightarrow \text{operator}$
 $* \rightarrow \text{operand 2}$

Eg: $t_2 = a + 1$

Diagram showing the mapping of the expression $t_2 = a + 1$ to its components:

- t_2 is labeled as "result".
- a is labeled as "operand 1".
- $+$ is labeled as "operator".
- 1 is labeled as "operand 2".

Eg:

$$x = a + b * c$$

TAC:

$$t_1 = b * c$$

$$t_2 = a + t_1$$

$$x = t_2$$

Eg: $x = a + b * c - d$

$$= a + (b * c) - d$$

$$t_1 = b * c$$

$$t_2 = a + t_1$$

$$t_3 = t_2 - d$$

$$x = t_3$$

→ TAC for Conditional Statements:

Consider:

if ($a < b$)

$x = a;$

else

$x = b;$

Three-address Code:

if $a < b$ goto L1
goto L2

L1:

$x = a$

goto L3

L2:

$x = b$

L3:

here,

- L1 → true branch
- L2 false branch
- L3 end of if-else

→ TAC for while loop:

Consider:

```
while(i < 10)
```

```
{
```

```
    i = i + 1;
```

```
}
```

TAC:

L1:

```
if i < 10 goto L2  
goto L3
```

L2:

```
t1 = i + 1
```

```
i = t1
```

```
goto L1
```

L3:

→ Represents of Three-Address Code:

Three Address Code

- Quadruples
- Triples
- Indirect Triples

(A) Quadruples:

A quadruples has four fields:

(operator, argument 1, argument 2, result)

Eg TAC:

$$t_1 = b * c$$

$$t_2 = a + t_1$$

$$x = t_2$$

Quadruple representation:

operator	Arg 1	Arg 2	Result
*	b	c	t ₁
+	a	t ₁	t ₂
=	t ₂	-	x

(B) Triples:

It has three fields:

(Operator, Arg1, Arg2)

here, there is no result.

eg: TAC:

$$t_1 = b * c$$

$$t_2 = a + t_1$$

$$x = t_2$$

Triples representation

Operator	Arg1	Arg2
*	b	c
+	a	(0)
=	(1)	x

here,

(0) means → result produced by inst 0.

(1) means → result produced by inst 1.

Note : v

Quadruples: Explicit result field

Tuple: result identified by instⁿ number.

(C) Indirect Triples :

Indirect triples use an additional pointer table.

Instead of directly referring to tuple positions, pointers refer to other tuples.

Pointer table

P0 → 0
P1 → 1
P2 → 2



Triples

0: *bc
1: +a(0)
2: (1) x

{ here, order is same as in triples (operation, arg1, arg2) }

The pointer table make rearranging the order of instⁿ easier.

Eg: ~ find quadruples, Triples and indirect triples. of:

$$x = (a+b) * (c-d)$$

⇒

TAC: ~

$$t_1 = a + b$$

$$t_2 = c - d$$

$$t_3 = t_1 * t_2$$

$$x = t_3$$

Quadruples representation:

operator	Arg 1	Arg 2	Result
+	a	b	t ₁
-	c	d	t ₂
*	t ₁	t ₂	t ₃
=	t ₃	-	x

Triples representation:

operator	Arg 1	Arg 2
+	a	b
-	c	d
*	(0)	(1)
=	(2)	x

Indirect Triples:

Pointer table

P0 : 0
P1 : 1
P2 : 2
P3 : 3



Triples

0 : + ab
1 : - cd
2 : * (0) (1)
3 : = (2) x

Page No.			
Date			

$$(a+b)/(c+d) * (a+b/c) - d$$

$\Rightarrow$ into quadruples, triples and indirect triples.

Given expression:

$$(a+b)/(c+d) \neq (a+b/c) - d$$

→ TAC [Three address code] : ~

$$t_1 = (a+b)$$

$$t_2 = (C + d)$$

$$t_3 = t_1 / t_2$$

$$t_4 = b/c$$

$$t_5 = a + t_4$$

$$t_6 = t_3 \times t_5$$

~~$$t_7 = t_6 - d$$~~

Quadruples Representation:

Operator	Arg 1	Arg 2	Result
+	a	b	t ₁
+	c	d	t ₂
/	t ₁	t ₂	t ₃
/	b	c	t ₄
+	a	t ₄	t ₅
*	t ₃	t ₅	t ₆
-	t ₆	d	t ₇

Triples Representation:

Operator	Arg 1	Arg 2
+	a	b
+	c	d
/	(0)	(1)
/	b	c
+	a	(3)
*	(2)	(4)
-	(5)	d

Indirect Triples:

Pointer table

P0 : 0
P1 : 1
P2 : 2
P3 : 3
P4 : 4
P5 : 5
P6 : 6



Triples:

0 : + a b
1 : + c d
2 : / (0) (1)
3 : / b c
4 : + a (3)
5 : * (2) (4)
6 : - (5) d

Q. Translate the expression :

$a = (a+b) * (c+d) + (a+b+c)$ into
(i) Quadruple (ii) Tree (iii) Indirect Tree

⇒ Given Expression :

$$a = (a+b) * (c+d) + (a+b+c)$$

TAC [Three Address Code] :-

$$t_1 = a + b$$

$$t_2 = c + d$$

$$t_3 = t_1 * t_2$$

$$t_4 = t_1 + c$$

$$[t_1 = a + b]$$

$$t_5 = t_3 + t_4$$

$$a = t_5$$

Quadruple Representation:

operator	Arg1	Arg2	Result
+	a	b	t ₁
+	c	d	t ₂
*	t ₁	t ₂	t ₃
+	t ₁	c	t ₄
+	t ₃	t ₄	t ₅
=	t ₅	-	a

Triples Representation:

Operator	Arg 1	Arg 2
+	a	b
+	c	d
*	(0)	(1)
+	(0)	c
+	(2)	(3)
=	(4)	a

Indirect Triples

Pointer table

P0: 0

P1: 1

P2: 2

P3: 3

P4: 4

P5: 5



Triples

0: + a b

1: + c d

2: + (0) (1)

3: + (0) c

4: + (2) (3)

5: =(4) a

- TAC (Three Address Code) Come after DAG
- TAC works on the principle of DAG (to optimize).

→ DAG :

DAG stands for Directed Acyclic Graph.

In Compiler Design, a DAG is an intermediate representation used mainly to represent expression inside a basic block and detect common subexpression.

→ Why do we need DAG?

Consider:

$$a = b + c$$

$$d = b + c$$

≠

$b + c$ is calculated twice.

A compiler can notice that both expression are the same and calculate it only once.

$$t_1 = b + c$$

$$a = t_1$$

$$d = t_1$$

This is called common subexpression elimination.

A DAG helps the compiler identify this common subexpression.

→ Meaning of DAG :-

• Directed :

Edges have a direction.

$b \rightarrow +$

$c \rightarrow +$

• Acyclic :

There is no cycle in the graph.

We cannot start at a node and follow arrows and eventually come back to the same node.

• Graph :

It consists of nodes and directed edges.

→ DAG Representation of an Expression :-

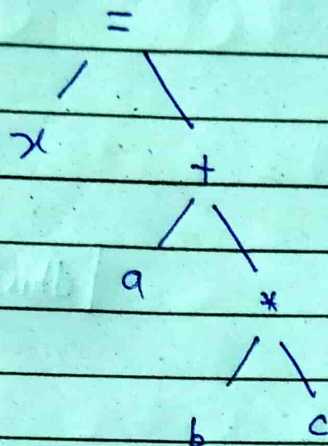
Consider :

$$x = a + b * c$$

→ first, according to operator precedence:

$$x = a + (b * c)$$

Page No.
 Date
 DAG:



→ DAG & Syntax tree difference :v

A syntax tree represents the structure of an expression.

where as,

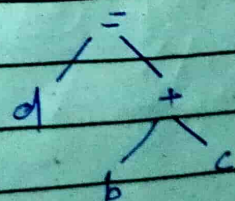
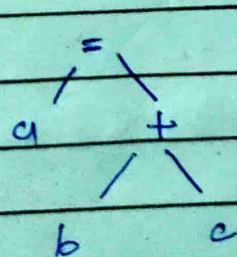
A DAG can share common subexpression.

Eg;

$$a = b + c$$

$$d = b + c$$

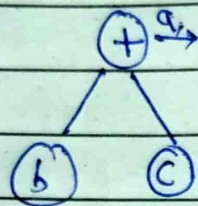
→ Without sharing:



The same expression appear twice

DAC:

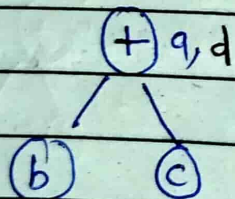
$$a = \underline{b+c}$$



[this (+) labels a]

$$d = \underline{b+c}$$

twice occurred



Q. Consider the following segment:

$$t_1 = a + b$$

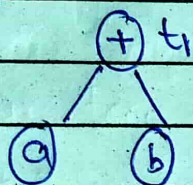
$$t_2 = c \times d$$

$$t_3 = a + b$$

$$t_4 = t_1 + t_2$$

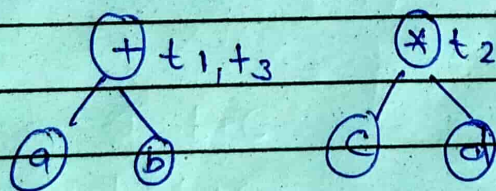
$\Rightarrow$

$$t_1 = a + b$$

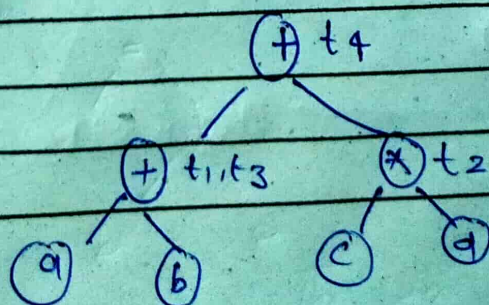


$$t_2 = c \times d$$

$$t_3 = a + b$$



$$t_4 = t_1 + t_2$$



Q. Construct DAG of following segment

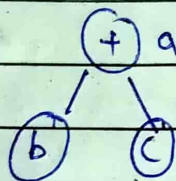
$$a = b + c$$

$$d = a - e$$

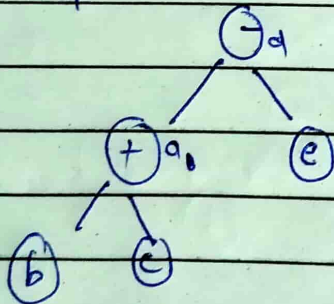
$$f = b + c$$

⇒

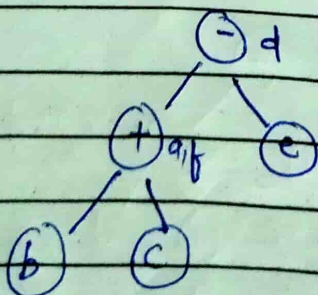
$$a = b + c$$



$$d = a - e$$



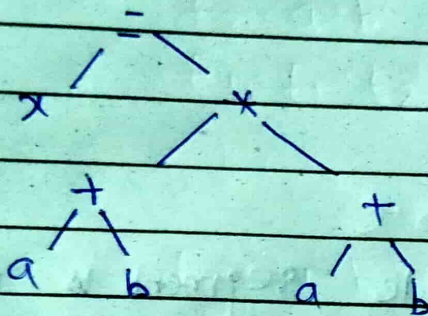
$$f = b + c$$



Q. $x = (a+b) * (a+b)$

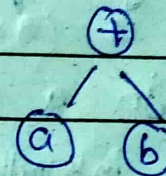
⇒

Syntax Tree

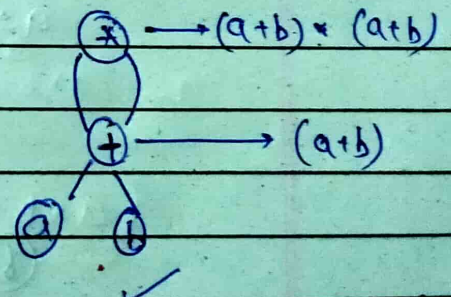


DAG

a+b



(a+b) * (a+b)



Q. Draw Syntax Tree and DAG for the Statement.

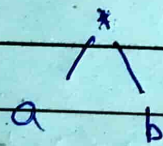
$$a = (a * b + c) \wedge (b + c) * b + c$$

$\Rightarrow$

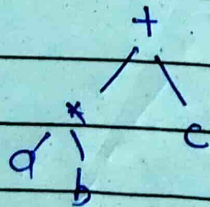
Syntax Tree :-

$$a = (a * b + c) \wedge (b + c) * b + c$$

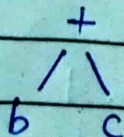
first: $a * b$



$(a * b + c)$

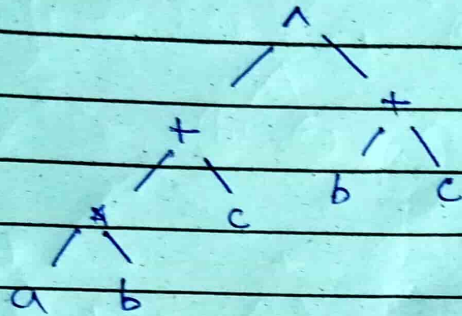


$(b + c)$

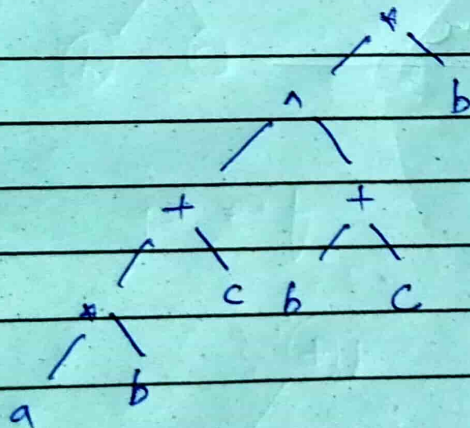


→ In DAG we do not repeat a single variable.
All variable in DAG always show at most one times.

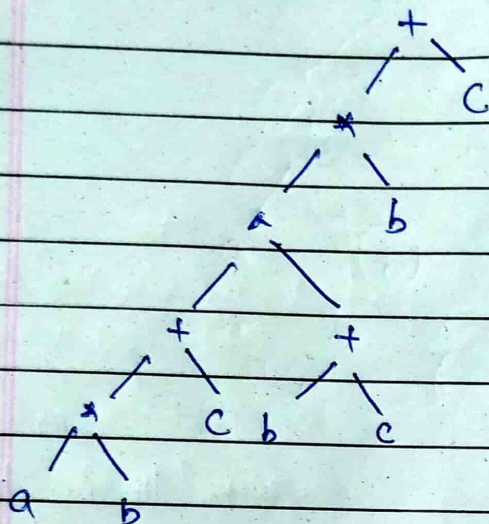
$$(a * b + c) \wedge (b + c)$$



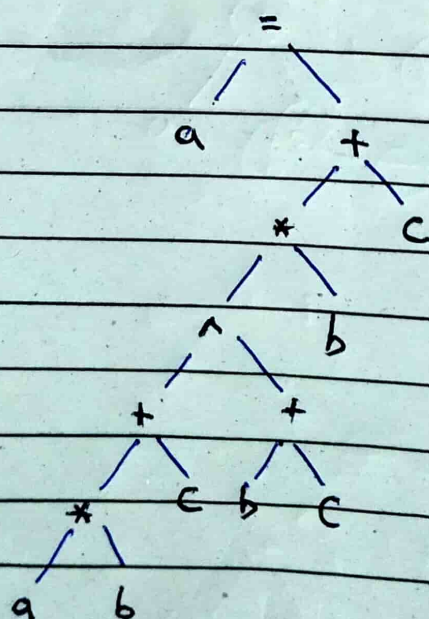
$$(a * b + c) \wedge (b + c) * b$$



$$(a * b + c) \wedge (b + c) * b + c$$



$$a = (a * b + c) \wedge (b + c) * b + c$$



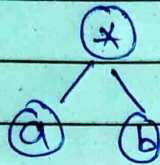
This is syntax tree.

DAG: No need to repeat the variable of the statement.

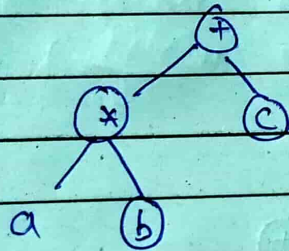
Page No.	Initial		
Date			

DAG: iv

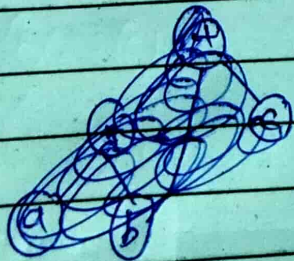
$a \times b$



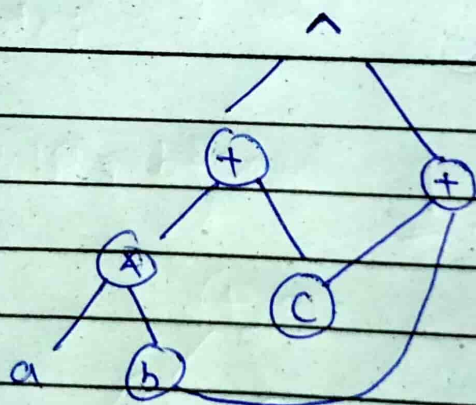
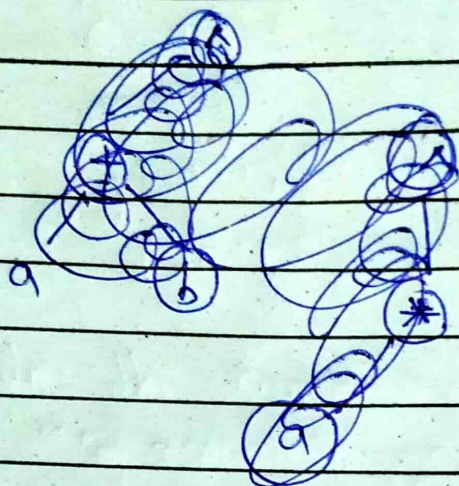
$(a \times b + c)$



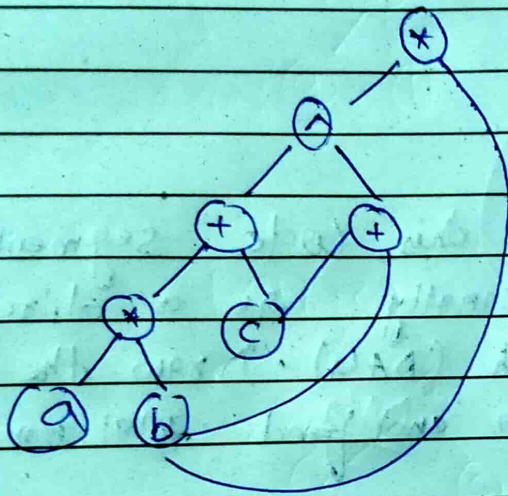
~~Ques~~ ~~scribble~~



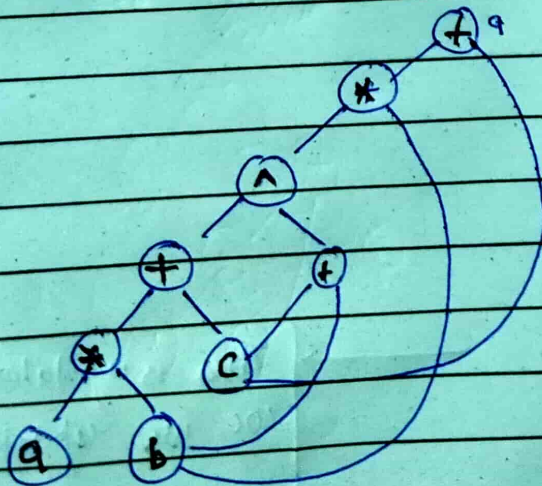
$$(a * b + c) \wedge (b + c)$$



$$(a \times b + c) \wedge (a + c) * b$$



$$(a \times b + c) \wedge (a + c) * b + c$$



This is DAG

Total Node = 9

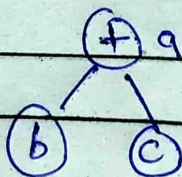
Q. Consider the following C code segment:

```
a = b + c;
c = a + 1;
d = b + c;
f = d + 1;
g = e + f;
```

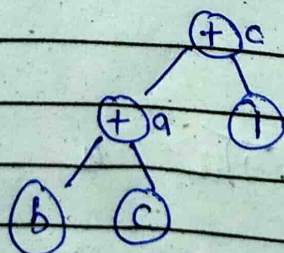
In the compiler, this code segment is represented internally as a directed acyclic graph (DAG). Draw the DAG representation and find total no. of nodes:

⇒

$a = b + c$

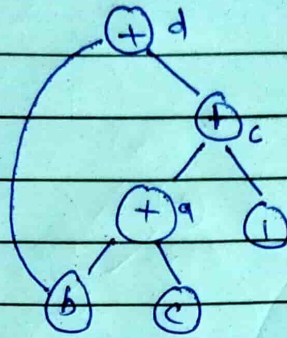


$c = a + 1$

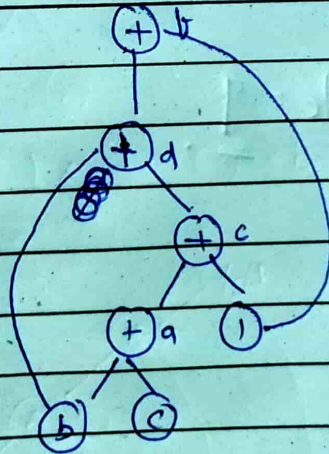


This is updated c , so we use updated c from now onwards.

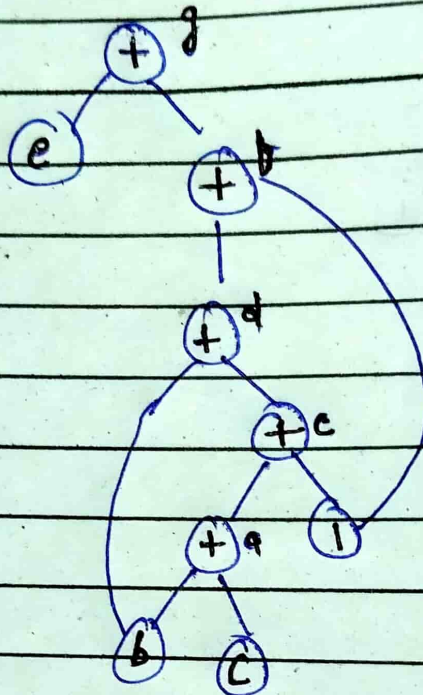
$$d = b + c$$



$$f = d + 1$$



$$g = e + f$$



DAG

Total no. of Node = 9

Akash